



Bounded Context:

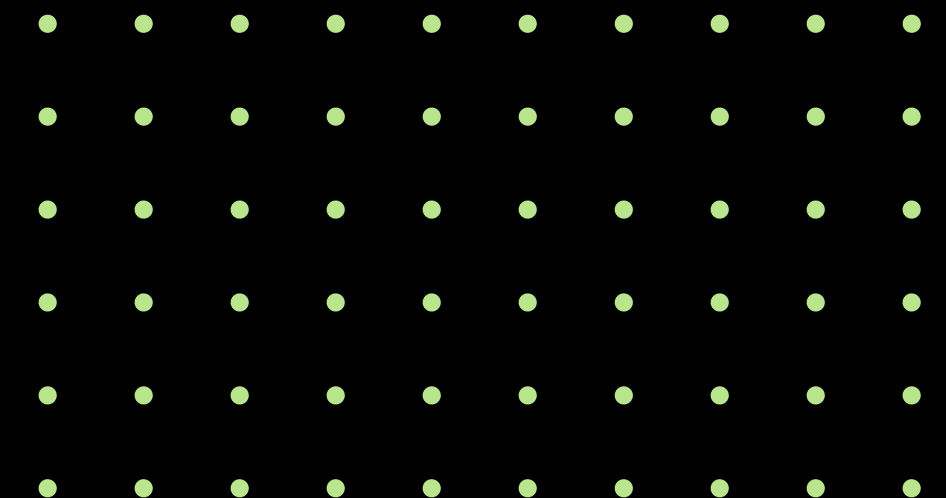
el arte de dividir el código



La pregunta del millón

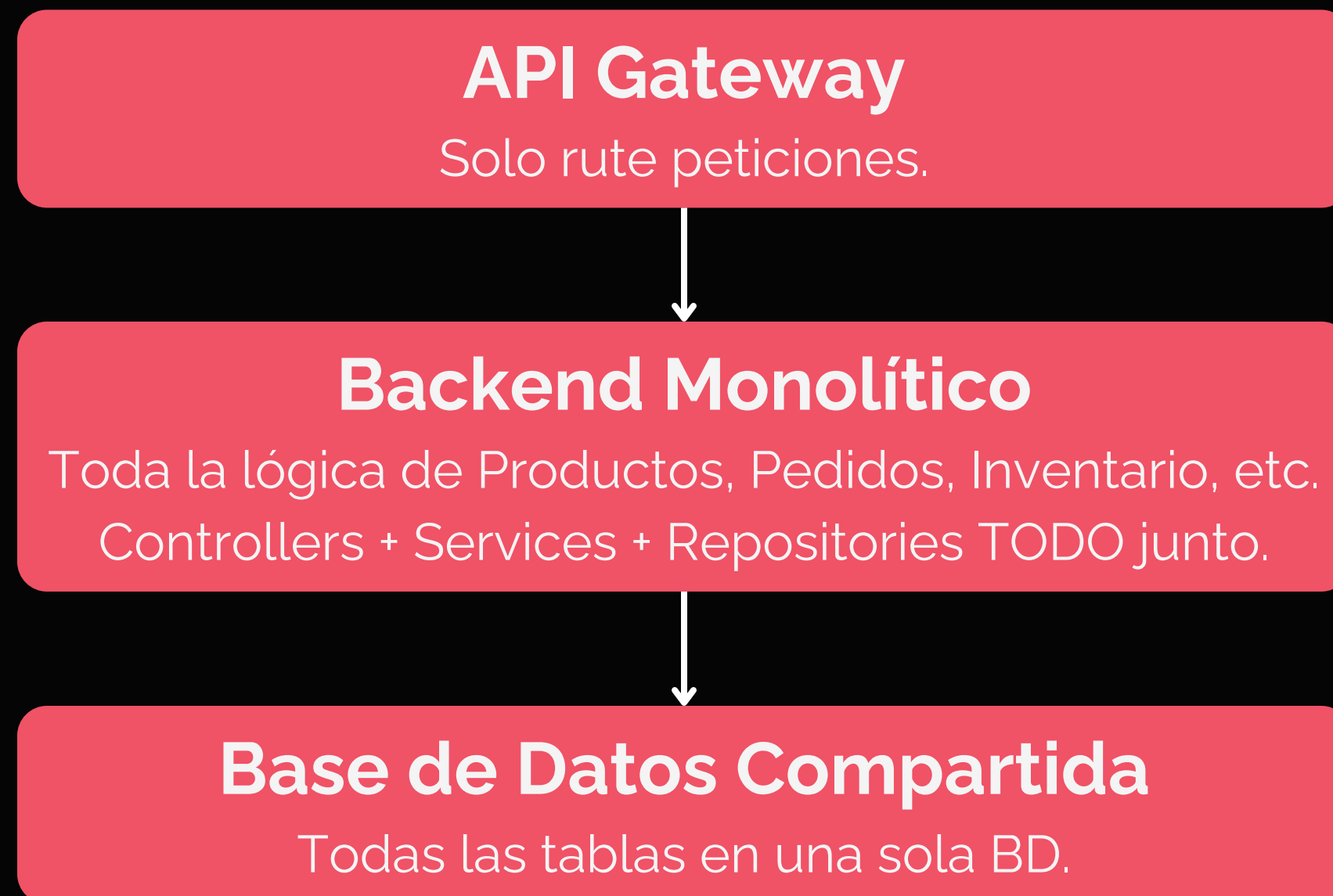


¿Cómo divido mi sistema?



Anti-patrón: división horizontal

Crear microservicios pensando en capas técnicas.



¿Por qué está mal?

- No hay independencia de despliegue. ✗
- No hay soberanía de datos. ✗

División vertical (Por funcionalidad)

Product service (completo)

Controllers + Services + Repositories

product_db:

- products.
- categories.
- product_images.

Inventory service (completo)

Controllers + Services + Repositories

inventory_db:

- stock_items.
- warehouses.
- stock_movements.

Order service (completo)

Controllers + Services + Repositories

order_db:

- orders.
- order_items.
- shipping_address.

¿Por qué está bien?

- Despliegue 100% independiente. 
- Soberanía de datos completa. 

Las reglas de oro son:

Despliegue independiente



Un microservicio debe poder ir a producción sin coordinar con nadie más.

Soberanía de datos



En Microservicios, está PROHIBIDO compartir la base de datos.





Reglas de oro

- *Un microservicio debe ser **VERTICAL**.*
- *Controllers, Services, Repositories) + **su propia base de datos COMPLETA**.*

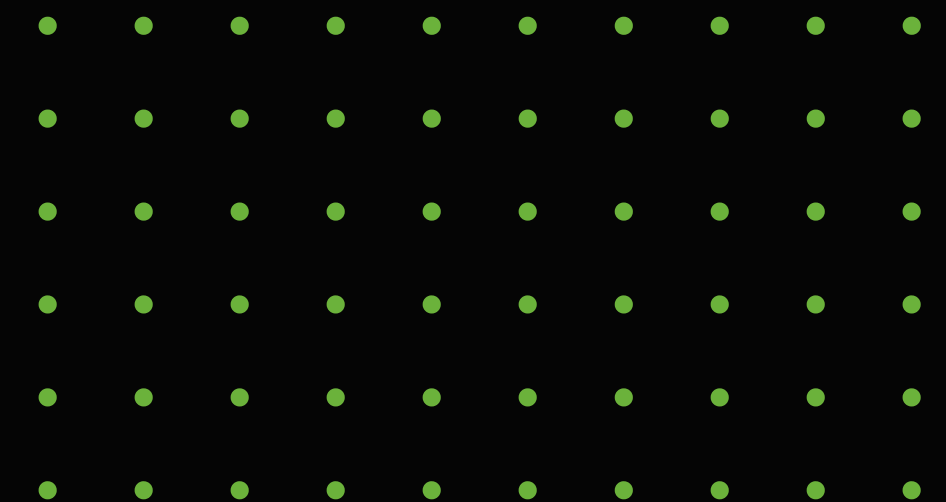
1 Microservicio = 1 BD con MÚLTIPLES tablas de ese dominio.





El problema del modelo único

Por: Gabriel Chaldú



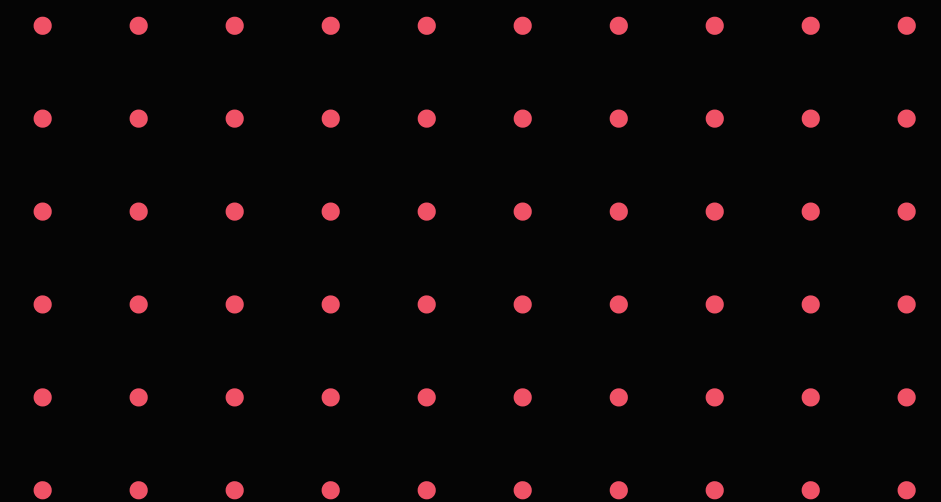
Un clase gigante



```
public class Product{  
    // For Sales  
    private String name;  
    private BigDecimal price;  
    private String imageUrl;  
  
    // For Inventory  
    private Integer currentStock;  
    private String aisleLocation;  
  
    // For Accounting  
    private Long supplierId;  
    private BigDecimal purchaseCost;  
}
```

¿Qué pasa aquí?

- El equipo de Inventario cambia el campo *ubicacionPasillo*.



DDD nos dice: rompe esa clase

Domain Driven Design (DDD)

Una clase gigante → Tres clases pequeñas y enfocadas.

Microservicio - Ventas

```
class Product {  
    String sku;  
    String name;  
    BigDecimal price;  
    String imageUrl;  
}
```

Microservicio -Inventario

```
class StockItem {  
    String sku;  
    Integer quantity;  
    String location;  
}
```

Microservicio - Contabilidad

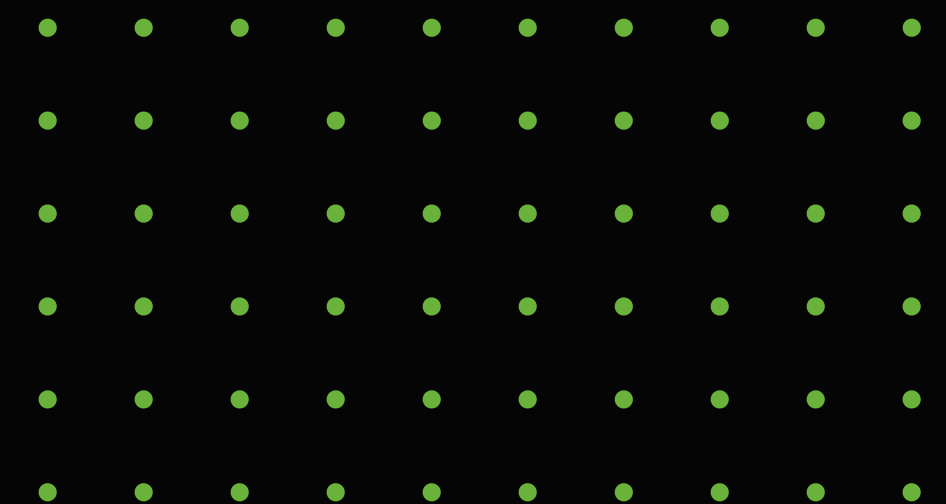
```
class PurchaseItem {  
    String sku;  
    BigDecimal cost;  
    Long supplierId;  
}
```



Bounded Contexts

y el ID como puente

Por: Gabriel Chaldú

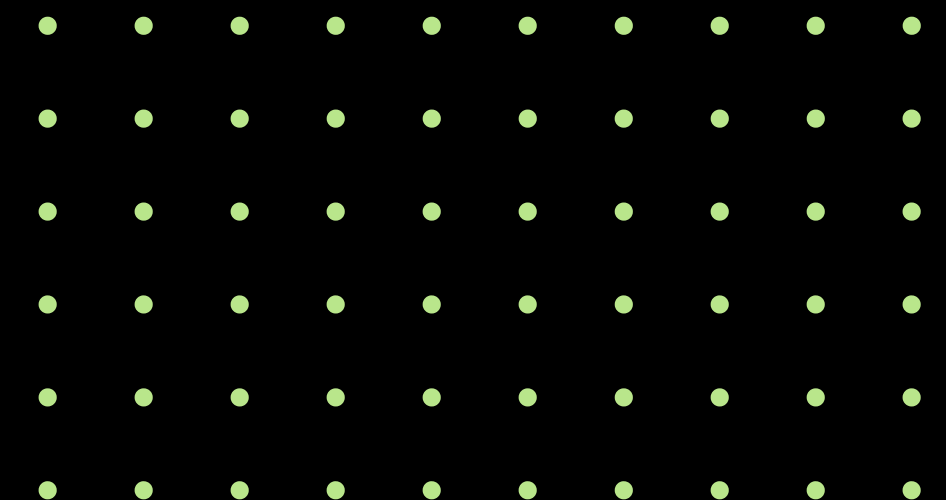


La duda del millón



Si los separamos en servicios diferentes...

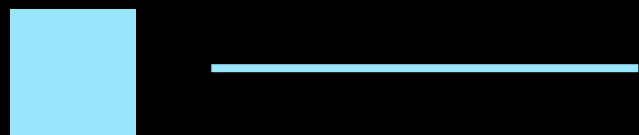
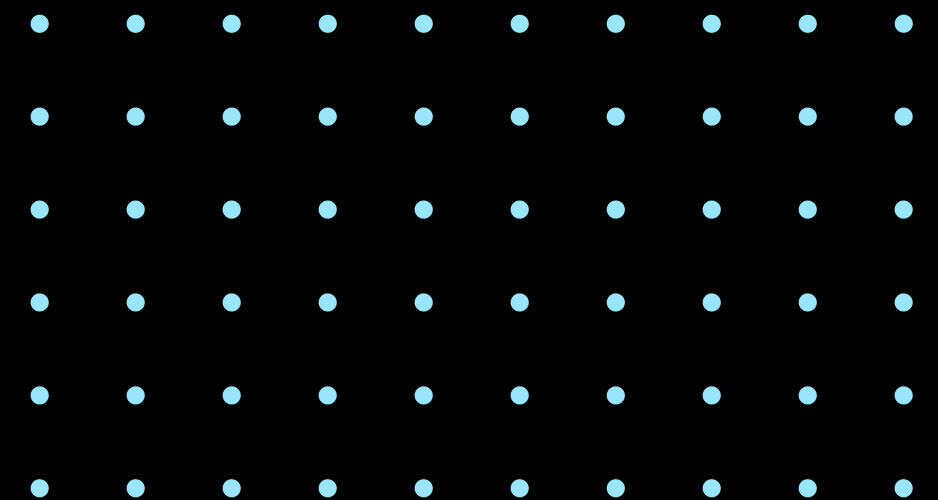
¿Cómo se conectan?



La respuesta



El Identificador Compartido (SKU).



El Identificador Compartido

product-service (Contexto catálogo)



```
class Product {  
    String sku; // "NIKE-001"  
    String nombre;  
    String descripcion;  
    String urlImagen;  
    // No sabe nada de stock  
}
```



SKU



inventory-service (Contexto inventario)



```
class InventoryItem {  
    String sku; // "NIKE-001"  
    Integer cantidad;  
    String fechaReposicion;  
    // No sabe nada de imágenes  
}
```

La palabra clave: Client



```
// 1. Cliente solicita producto
Product product = productService.getProduct("NIKE-001");

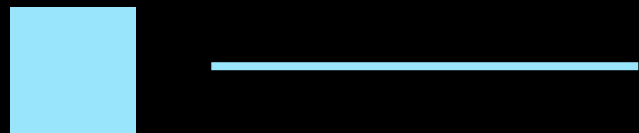
// 2. Consultar stock en otro servicio
Integer stock =
inventoryClient.getStock(product.getSku());
// 3. Combinar la información
return new ProductWithStock(product, stock);
```



¿Qué es un Bounded Context?



"...es un límite donde una palabra tiene un significado según su contexto."





3 Reglas de oro

- *Separamos verticalmente.*
- *Unimos por IDs.*
- *Datos aislados.*

Ejemplo concreto

product-service → produc_db



products:

- id.
- sku.
- name.
- price.



categories:

- id.
- name.



product_images:

- id.
- product_id.
- url.

inventory-service → inventory_db



stock_items:

- sku.
- quantity.
- location.



warehouses:

- id.
- name.
- address.



stock_movements:

- id.
- sku.
- quantity.
- date.

{dev/talles}



**¡A seguir
mejorando!**

